Тема 2. Базовые типы данных языков программирования высокого уровня

Лекция 4

Базовые типы данных языков программирования высокого уровня (часть 2)

Как указывалось ранее, для отображения особенностей представления в компьютере данных различной природы в информатике, в компьютерных дисциплинах используется важнейшая *концепция типа данных*

Тип данных представляет собой важнейшую характеристику, которая определяет:

- множество допустимых значений;
- множество операций, которые могут выполняться над значением;
- структуру значения (скаляр, вектор и т.д.);
- способ машинного представления значения.

Основными принципами концепции типа данных в языках программирования являются следующие положения:

- Тип константы, переменной или выражения может быть определен по внешнему виду (по изображению) или по описанию без выполнения каких-либо вычислений.
- Любая операция или функция требует аргументов и возвращает результат вполне определенного типа. Типы аргументов и результатов операций определяется по вполне определенным правилам языка.

2.6. Конструируемые типы данных

В информатике используется большое количество различных *типов*, различных *структур данных*, которые применяются для моделирования объектов, встречающихся в рассматриваемых задачах. Значение *скалярного (простого, атомарного)* типа представлено ровно одним компонентом (пример: время, температура). Значение *структурированного (составного)* типа представлено более чем одним компонентом (пример: вектор, матрица, таблица и т.д.). ***Структурированные (составные) типы*** характеризуются: количеством и возможным типом компонентов значения, а также способом доступа к отдельному компоненту значения.

Рассмотрим группы разновидностей типов данных, которые в литературе часто называют "составными", поскольку любое значение любого из этих типов состоит из значений одного или нескольких других типов. Более предпочтительным является термин "конструируемый тип", поскольку для каждой разновидности типов этой группы в языке программирования специфицируются средства построения (конструирования) нового типа на основе встроенных и/или ранее определенных типов, и для каждой разновидности предопределяются операции, позволяющие извлечь компонент составного значения. К наиболее распространенным конструируемым типам относятся тип массива, тип строки, тип записи и тип множества.

2.6.1. Массивы

Решение многих задач упрощается, если набор однотипных объектов хранить в виде последовательности соседних ячеек памяти. Доступ к этим ячейкам обычно осуществляется посредством индекса.

Структуры аналогичные векторам и матрицам в информатике принято называть ***массивами.*** Все элементы массива должны быть одного и того же типа.

$$\bar{x} = \{x_1, x_2, x_3\}; \quad \bar{w} = (w_1, w_2, w_3, w_4, w_5, w_6, w_7, w_8, w_9); \quad A = \begin{bmatrix} a_{11} a_{12} a_{13} \\ a_{21} a_{22} a_{23} \\ a_{31} a_{32} a_{33} \end{bmatrix}$$

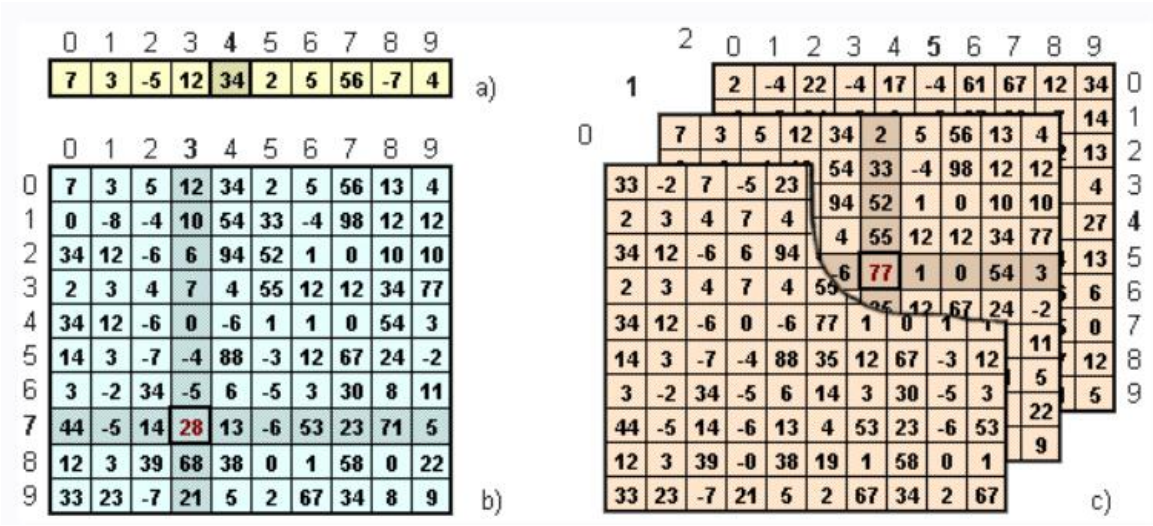
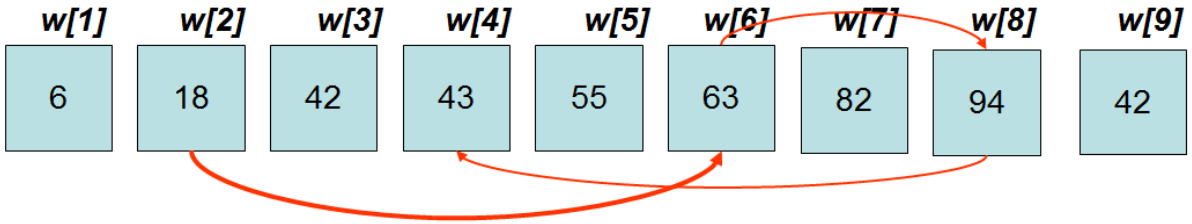


Рис. 2.6. Графическое представление массивов целого типа:
а) одномерный; б) двумерный; с) трехмерный

Для доступа (обращения) к отдельному элементу массива используется индекс или несколько индексов (**w[5]; w[i+2]; A[1,2]**). Индексы могут быть выражениями, значения которых могут произвольным образом изменяться в заранее заданных границах. Поэтому говорят, что к элементам массивов имеется ***прямой доступ***.



В языке C++ под индексом элемента массива понимают смещение относительно адреса начала массива в оперативной памяти, выраженное в элементах (рис. 2.6). В связи с этим в C++ принята модель, в соответствии с которой описание массива, по существу, сводится к описанию адресной

переменной (указателя), хранящей адрес первого элемента массива (с индексом 0).

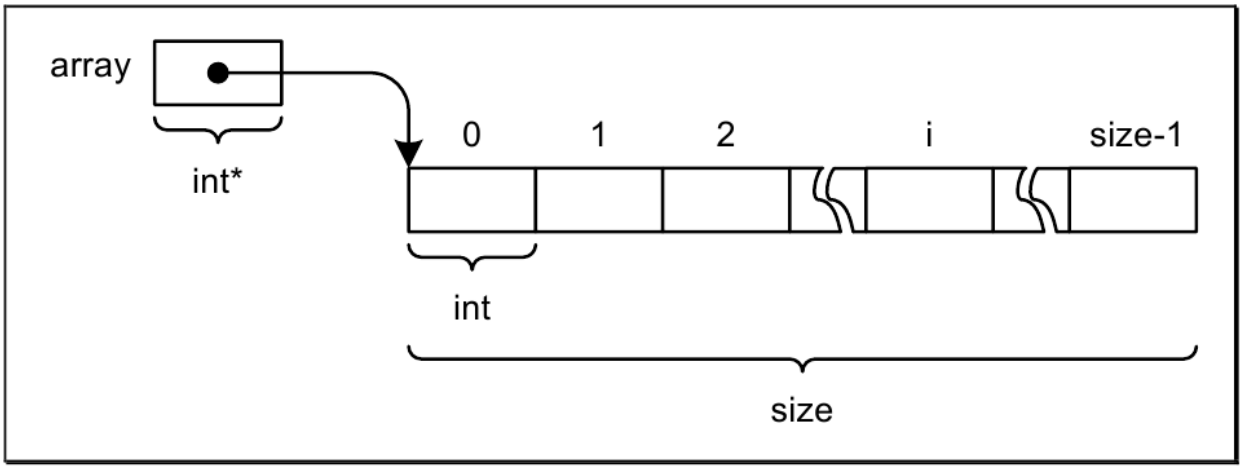


Рис. 2.6. Массивы в C/C++

Таким образом, в C++ индекс может изменяться от 0 до size-1, где size – число элементов в массиве. Отметим, что компилятор не имеет возможности контролировать ошибки индексирования при обращении к элементу массива. Ошибки индексирования (равно как и ошибки при работе с неинициализированными или некорректно инициализированными указателями) являются наиболее неприятными в связи с трудностью их обнаружения.

Отметим, что другие языки могут реализовывать модель индексации, отличную от модели C++. Например, в языке Pascal индекс элемента массива по умолчанию изменяется в интервале 1..size. Программист может определить и другой диапазон изменения индекса (включая отрицательные значения индексов и использование в качестве индекса значений других порядковых типов).

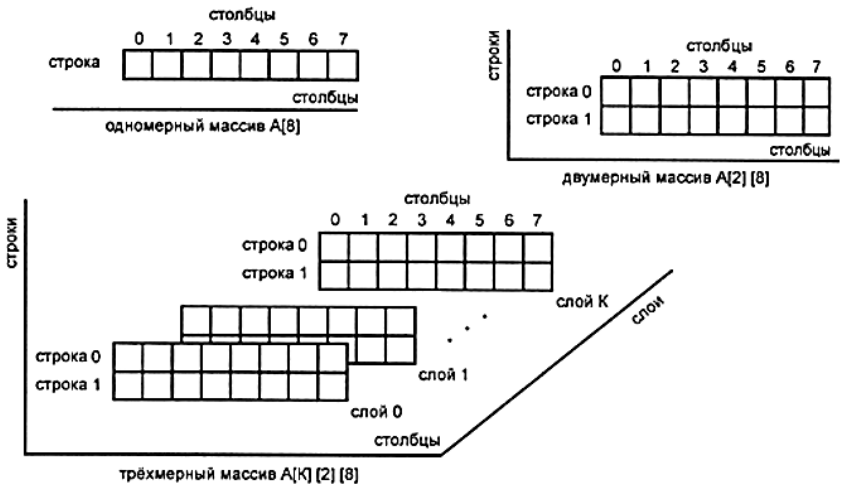
Фактически, индекс массива в языке Pascal, является объектом данных типа «диапазон», жестко фиксирующего ограниченное множество значений, которые могут принимать элементы этого типа. Модель массива, реализованная в языке Pascal, не предусматривает явной работы программиста с адресной информацией. За счет промежуточного уровня

абстрагирования компилятор Паскаля обеспечивает строгий контроль корректности индексирования.

Перечень операций над массивами в существенной степени зависит от того, как реализована абстракция массива в языке программирования. В языке С любая операция над массивом в целом (например, копирование одного массива в другой) сводится к последовательности операций над отдельными элементами. При этом ответственность за корректное использование памяти, отведенной под массивы, полностью лежит на программисте.

Ваныкина Г.,Сундукова Т. Структуры и алгоритмы компьютерной обработки данных
http://www.intuit.ru/studies/courses/648/504/lecture/11433?page=2

Массив – именованная последовательность областей памяти, хранящих однотипные элементы. Каждая такая область памяти называется *элементом массива*. Массивы обладают размерностью (большей или равной единице), которой задается число элементов, содержащихся в них, а также измерением, что предполагает возможность описания в программе одно- и многомерных массивов. Количество элементов в массиве называется его *размером*.



Простейшим аналогом двумерного массива может быть *таблица*, а трехмерного – несколько таблиц одинакового размера. Математические

объекты типа *вектор* и *матрица* – примеры аналогов (соответственно одно- и двумерных) массивов.

Тип элемента массива может быть одним из базовых (*скалярных*), *типом* другого массива, типом указателя, типом структуры или объединения.

Элементы массива в C++ нумеруется, начиная с нуля. У одномерных массивов после его имени указывается один *индекс* (порядковый номер), заключенный в прямоугольные скобки [ ], а у многомерных – несколько, каждый из которых заключается в [ ]. Последнее означает, что *многомерный массив* создается путем определения массива из элементов типа *массив*.

Все элементы массива имеют одно имя – имя массива и отличаются индексами – порядковыми номерами в массиве. В определении массива можно задать его *размерность* по каждому измерению. Допустимо явное задание массива либо с помощью указателя (объекта, хранящего *адрес* начала области набора значений). Резервирование памяти для массива выполняется на этапе компиляции программы.

При объявлении массива *компилятор* выделяет для него последовательность ячеек памяти, для обращения к которым в программе применяется одно и то же имя. В то же время *массив* позволяет получить *прямой доступ* к своим отдельным элементам.

Объявление одномерных массивов

Синтаксис определения массива без дополнительных *спецификаторов* и модификаторов имеет два формата:

Тип ИмяМассива[ВыражениеТипаКонстанты];

Тип ИмяМассива[];

ИмяМассива – *идентификатор* массива.

Тип – тип элементов объявляемого массива. Элементами массива не могут быть функции, файлы и элементы типа void.

ВыражениеТипаКонстанты – задает количество элементов (*размерность*) *массива. Выражение* константного типа вычисляется на

этапе компиляции. Данное константное выражение может быть опущено в случаях если:

- при объявлении массив инициализируется;
- массив объявлен как *формальный параметр* функции;
- массив объявлен как ссылка на массив, явно определенный в другом файле.

Например:

1. int a[100]; //массив из 100 элементов целого типа
2. double d[14]; // массив из 14 элементов типа double
3. char s[]="Программирование"; //символьный массив
4. const int t=5, k=8;
   float wer[2*t+k];
   //массив из 2*t+k элементов вещественного типа
5. int sample[853];
   /*массив из элементов sample[0], sample[1],
   sample[2],...,sample[852] типа int*/

   равносильно объявлению

   const int N_max=853;
   int sample[N_max];

   равносильно объявлению

   #define N_max 853
   ...
   int sample[N_max];

В языке C++ не производится проверки границ массивов: таким образом, *исполнение* кода не остановится при выходе за границы массива. Если *переполнение* массива происходит во *время выполнения оператора присваивания*, то лишние значения могут присвоиться другим переменным или включиться в текст программы. С другой стороны, можно объявить *массив* размером *N* и указать *индекс* элемента, выходящий за пределы *N*, что не приведет к появлению сообщений об ошибке, как на шаге компиляции, так и на шаге выполнения, даже если

это послужит причиной аварийного завершения программы.

Инициализация одномерных массивов

Термином "*инициализация*" обозначают возможность задать начальные значения элементов массива без программирования соответствующих действий. Например, не прибегая к программным средствам типа присваивания значений в цикле или считывания данных из внешнего источника (*файл*, клавиатура, *блок данных*).

В C++ одновременно с объявлением массива можно задать начальные значения всех элементов массива или только нескольких первых его *компонент*.

Например:

```
float t[5]={1.0, 4.3, 8.1, 3.0, 6.74};
char b[7]={'П','р','и','в','е','т'};
/*в данных примерах длину массива компилятор вычисляет по
количеству начальных значений, перечисленных в фигурных
скобках*/
int d[10]={1, 2, 3};
char a[10]="Привет";
/*в данных примерах определяется значение только заданных
переменных d[0],d[1],d[2] и a[0],a[1],...,a[9], остальные
элементы не инициализируются*/
```

Если в определении массива явно указан его размер, то количество начальных значений не может быть больше количества элементов в массиве.

Метод инициализации безразмерных массивов не только менее трудоемок, но и позволяет заменить любое сообщение, без перерасчета размера соответствующего массива.

Краткие итоги

1. Массив является представителем *структурированного типа данных* в языке C++.
2. Элементы массива имеют одинаковые имя, тип и располагаются в

памяти последовательно.

3. Элементы массива характеризуются индексами, значениями и адресуемой памятью.
4. Существует две основные формы объявления массивов: с указанием и без указания размера. Безразмерный массив объявляется, если: он инициализируется при объявлении, является *формальным параметром* функции, объявлен как ссылка на массив.
5. Инициализация массива заключается в присваивании начальных значений его элементам. В программах часто используется генерация массивов.
6. Для массивов нельзя выполнить операцию прямого присваивания.
7. Адресация элементов массива осуществляется с помощью индексированного имени. Обращаться к элементам массива можно также посредством механизма указателей.
8. Размер памяти, занимаемой массивом, зависит от реализации и вычисляется с помощью *операции sizeof*.
9. Массивы используются для решения прикладных задач.

2.6.2. Строки

Наиболее типичным использованием массива являются строки. *Строкой* называется последовательность символов. Строка – это динамическая структура. В процессе выполнения программы количество элементов строки (ее длина) может изменяться от нуля до максимального размер памяти, выделенной под эту строку. Практически, строка представляет собой массив символов из элементов типа *char*.

Доступ к элементам строки, также как и к элементам массива – прямой. Элементы, начиная с первого содержат символы строки, количество которых совпадает с динамической длиной строки.

К операциям над строками относятся присваивание, сравнение и конкатенация.

Операция присваивания. Операндами могут быть символы, строки, символьные массивы. Результатом операции является строка равная значению операнда. Если строковой переменной присваивается значение, превышающее её длину, то перед присваиванием происходит усечение присваиваемого значения.

Операция сравнения. Переменные строкового типа можно сравнивать между собой. Из двух строк является та большей, у которой первый из неравных символов больше (по ASCII-коду). Иначе они равны.

Операция конкатенация. Операндами могут быть символы, строки, символьные массивы. Результатом является строка, полученная дописыванием в конец первого операнда второго операнда.

Хотя С не определяет строкового типа, имеется поддержка строковых операций с помощью функций, имеющихся в данном языке. В С строка определяется как символьный массив произвольной длины, оканчивающийся нулевым символом. Нулевой символ определяется как ‘\0’. Поэтому необходимо объявлять символьные массивы на один символ больше, чем требуется для хранения самой длинной строки. Например, если необходимо объявить массив s, содержащий десятисимвольную строку, следует написать:

```
char s[11];
```

В результате этого выделяется место в конце строки для нулевого символа. Хотя С не имеет строкового типа данных, имеются строковые константы. Строковые константы – это список символов, заключенных в двойные кавычки. Ниже показаны две строковые константы:

```
“Hello”  “this is a test”
```

Не надо добавлять нулевой символ в конец строковой константы, поскольку компилятор С выполняет это автоматически. Приведем пример объявления символьного массива:

char str[] = "объявление символьного массива";

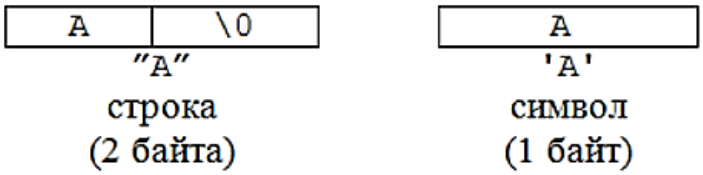
char str[6] = {'h','e','l','l','o','\0');

Ваныкина Г.,Сундукова Т. Структуры и алгоритмы компьютерной обработки данных
http://www.intuit.ru/studies/courses/648/504/lecture/11429

Строка – это последовательность символов, заключенная в двойные кавычки (" ").

Размещая строку в памяти, *транслятор* автоматически добавляет в ее конце символ '\0' (нулевой символ или нулевой *байт*, который является признаком конца строки). В записи строки может быть и один символ: "A" (заключен в двойные кавычки), однако, в отличие от *символьной константы* 'A' (используются *апострофы*), длина строки "A" равна 2 байтам.

В языке C++ строка – это пронумерованная последовательность символов (*массив символов*), она всегда имеет тип char[]. Все символы строки нумеруются, начиная с нуля. *Символ конца строки* также нумеруется – ему соответствует наибольший из номеров. Таким образом, строка считывается значением типа "*массив символов*". Количество элементов в таком массиве на 1 больше, чем изображение соответствующей строки, так как в конец строки добавлен нулевой символ '\0'.



Представление символа и строки

Присвоить *значение* строке с помощью *оператора присваивания* нельзя, так как для массивов не определена операция прямого присваивания. Поместить строку в *символьный массив* можно либо при

вводе, либо с помощью инициализации:

```
char s1[] = "ABCDEF"; //инициализация строки

char s2[]={'A','B','C','D','E','F','\0'};//инициализация строки
```

……

Краткие итоги

1. Для представления текстовой информации в C++ используются символьные данные и строки.
2. В C++ не определен *строковый тип* данных, и строка представляется как *массив символов*.
3. Инициализировать строку можно как *массив символов*.
4. Признаком конца строки является нулевой символ.
5. Обратиться к элементу строки можно по индексу, который соответствует порядковому номеру элемента.
6. Нумерация элементов строки начинается с нуля. Размер строки определяется количеством входящих в нее символов.
7. В C++ предусмотрены различные способы ввода и вывода одиночных символов и строк: с помощью стандартных функций, с помощью потокового или форматированного ввода/вывода. При считывании строки с клавиатуры признак конца строки добавляется автоматически.
8. Каждому символу однозначно соответствует его внутренний код.

2.6.3. Записи (Структуры)

Типы массивов позволяют работать с регулярными структурами данных, каждый элемент которых относится к одному и тому же базовому типу. Существует другая разновидность составных конструируемых типов данных, которые позволяют определять и использовать нерегулярные

структуры данных, элементы которых могут относиться к разным встроенным или явно определенным типам данных. Собирательно типы этой разновидности называются *типами записи* или *структурными типами*.

Идея состоит в том, что в определении структурного типа перечисляются имена полей записи, и для каждого поля указывается его тип данных. После этого можно определять переменные вновь сконструированного типа и производить доступ к полям переменных.

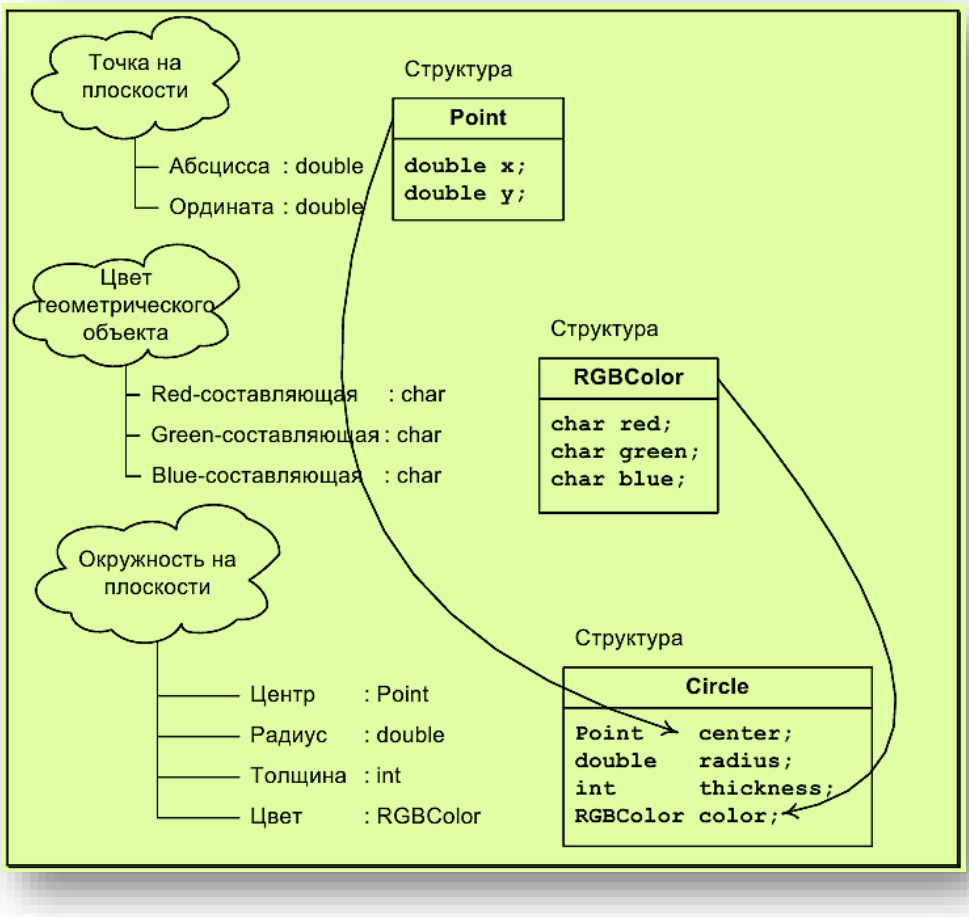


Рис. 2.7. Конструирование составных типов

Записи - структуры, аналогичные строкам таблицы. Компоненты записей принято называть ***полями***. Различные поля (столбцы таблицы) могут быть разных типов. Для доступа к отдельным полям записи используются их фиксированные и неизменные имена. Например: ***День Победы. Месяц := май***. Поля могут выбираться для обработки в произвольном порядке, поэтому говорят, что доступ к компонентам записи прямой.

	День	Месяц	Год
День Победы:	**9**	**май**	**1945**
Полёт Гагарина:	**12**	**апрель**	**1961**

Ваныкина Г.,Сундукова Т. Структуры и алгоритмы компьютерной обработки данных
http://www.intuit.ru/studies/courses/648/504/lecture/11439

Структура – это *составной объект*, в который входят элементы любых типов, за исключением функций. В отличие от массива, который является однородным объектом (все элементы относятся к одному типу данных), структура может быть неоднородной. Таким образом, структура – это *тип данных*, сформированный из объектов однородных либо разнообразных типов данных.

Структуру можно представить себе как *запись*, состоящую из нескольких полей или элементов. Структуры обеспечивают удобный способ организации связанных по смыслу *переменных*.

В некоторых языках программирования, в частности в *Pascal*, структуры называются записями. Структуры помогают в организации сложных данных (особенно в больших программах), поскольку позволяют группу связанных между собой переменных трактовать не как множество отдельных элементов, а как единое целое.

Традиционный пример структуры – строка *платежной ведомости*. Она содержит такие сведения о служащем, как его *полное имя, адрес*, номер карточки социального страхования, зарплата и т.д. Некоторые из этих характеристик сами могут быть структурами: например, *полное имя* состоит из нескольких *компонент* (фамилии, имени и отчества); аналогично *адрес* и даже зарплата. Другой пример – из области графики: точка на плоскости есть пара вещественных координат, шар в пространстве моделируется четырьмя *вещественными числами* и т. д.

Массив структур – это массив, каждый элемент которого является структурой. В памяти элементы массива структур размещаются последовательно.

Массивы структур широко используются для структурной организации данных в *прикладных* программах и системном программном обеспечении.

2.6.4. Множества

Во многих математических и информационных задачах возникает необходимость в прямом или косвенном использовании основного математического объекта **множества.**

Аналогично массивам, множества представляют собой набор однородных объектов, однако, в отличие от массивов, порядок следования элементов множества является несущественным. Если для массива основной операцией доступа к элементу является операция обращения по индексу, то для множества – это операция проверки принадлежности элемента множеству.

Соответствующий множеству тип данных по определению относится к структурированным, так как в общем случае множество может состоять более чем из одного элемента, и при этом со всеми элементами множества приходится выполнять операции как с единым целым. Над множеством определяют ряд специфических операций:

- количество элементов в множестве заранее не определяется, и с течением времени оно может изменяться
- все элементы множества должны быть одного и того же типа
- **доступа** к отдельным элементам множества **нет**; можно только узнать принадлежит элемент множеству или нет, включить элемент в множество или исключить его из множества

- предусмотрены стандартные операции над множествами: *объединение, пересечение, вычитание* и т.д.

Очевидно, что обычный одномерный массив можно использовать в качестве внутренней реализации множества, однако такая реализация может оказаться неэффективной, поскольку многие типичные для множества операции будут выполняться слишком долго. В программистской практике, однако, чаще всего встречаются не множества общего вида, а такие множества, которые могут быть сведены к множеству целых числе из некоторого диапазона (обычно не слишком большого). В этом случае наиболее эффективная реализация получается, если хранить не сами элементы множества, а только информацию о принадлежности данного элемента множеству. В основе такой реализации обычно лежит использование так называемой шкалы битов. Каждый бит и определяет, принадлежит элемент множеству или нет. Общее число битов в шкале равно числу всех возможных элементов множества (то есть мощности универсального множества для данной предметной области).

Средствами языка C++ можно представить битовую шкалу посредством массива слов одинаковой разрядности (в рассматриваемом ниже примере используются 16-разрядные слова, соответствующие типу unsigned short). Тогда для доступа к биту номер n следует сначала определить индекс слова (в нашем случае – это n/16), а затем определить конкретный бит в слове, вычислив остаток от деления на 16.

Соответствующие действия эффективно реализуются посредством операций поразрядного сдвига. Определив местоположение искомого бита, можно реализовать две базовые операции – добавление элемента (используется операция поразрядного ИЛИ) и проверка принадлежности элемента множеству (используется операция поразрядного И).

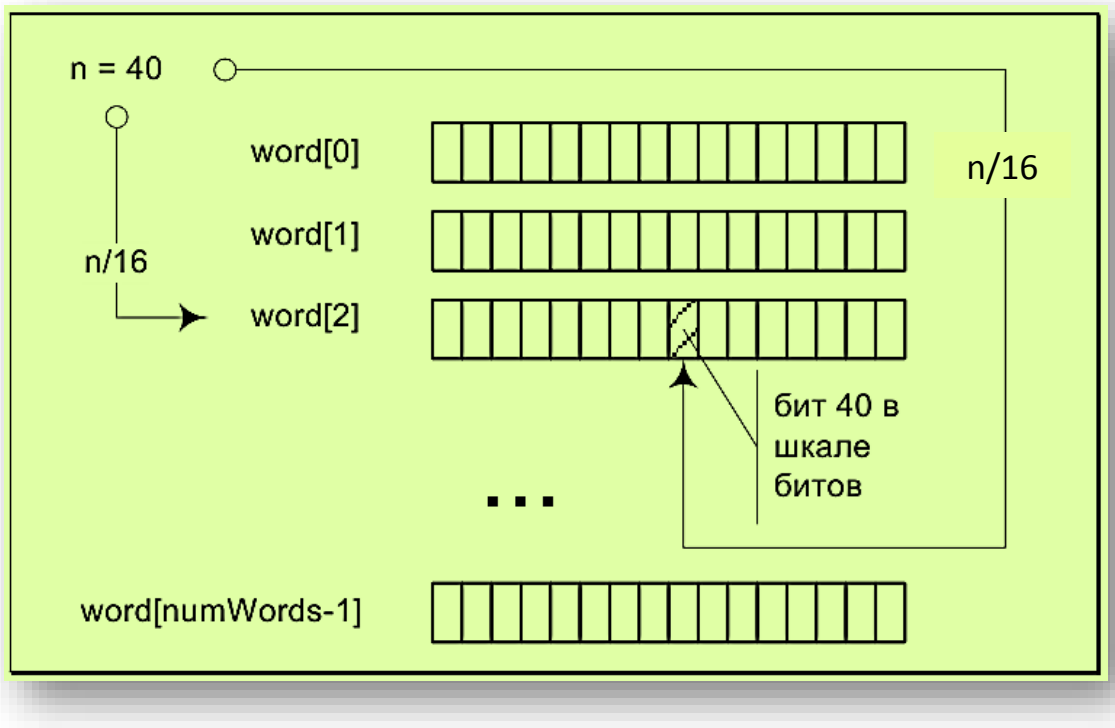


Рис. 2.8. Шкала битов

2.7. Указательные типы данных

Понятие указателя в языках программирования является абстракцией понятия машинного адреса. Подобно тому, как зная машинный адрес, можно обратиться к нужному элементу памяти, имея значение указателя, можно обратиться к соответствующей переменной. Различие между механизмами указателей в разных языках состоит главным образом в том, откуда берется значение указателя. Чем больше возможностей по работе с указателями, тем более эффективную программу можно написать и тем "опаснее" становится программирование. Обычно возможности оперирования указателями ограничиваются по мере повышения уровня языка и усиления его типизации. В любом случае для объявления указательных переменных служат так называемые ***указательные типы***.

В языках линии Паскаль переменной указательного типа можно присваивать только значения, вырабатываемые встроенной процедурой

динамического выделения памяти new, значения переменных того же самого указательного типа и специальное "пустое" ссылочное значение nil, которое входит в любой указательный тип. Не допускаются преобразования типов указателей и какие-либо арифметические действия над их значениями. С переменной-указателем можно выполнять только операцию, обеспечивающую доступ к значению переменной, на которую указывает значение переменной-указателя.

Напротив, в языках Си и C++ имеется полная свобода работы с указателями. С помощью операции "&" можно получить значение указателя для любой переменной, над указателями определены арифметические действия, возможно явное преобразование указательных типов и даже преобразование целых типов к указательным типам. В этих языках не фиксируется значение "пустых" (ни на что не ссылающихся) указательных переменных. Имеется лишь рекомендация использовать в качестве такого значения константу с символическим именем NULL, определяемую в библиотечном файле включения. По сути дела, понятие указателя в этих языках очень близко к понятию машинного адреса.

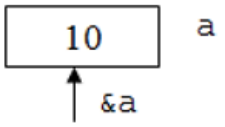
Подводя итоги этого краткого обсуждения механизма указателей в Си/C++, заметим, что позволяя программировать с очень большой эффективностью, этот механизм делает языки очень опасными для использования и требует от программистов большой аккуратности и сдержанности. При разработке получающего все большее распространение языка Java (одним из основных предков которого был C++) для повышения уровня безопасности были резко ограничены именно средства работы с указателями в языке C++.

Ваныкина Г.,Сундукова Т. Структуры и алгоритмы компьютерной обработки данных
http://www.intuit.ru/studies/courses/648/504/lecture/11425

Указатели являются специальными объектами в программах на C++. Они предназначены для хранения адресов памяти.

Рассмотрим пример. Когда компилятор обрабатывает оператор определения переменной, например, int **a**=10; , то в памяти выделяется

участок памяти в соответствии с типом переменной и записывается в этот участок указанное значение (размер типа int не менее 2 байтов и зависит от реализации). Все обращения к переменной **a** компилятор заменит на адрес области памяти, в которой хранится эта переменная. Операция **&a** является операцией взятия адреса ее операнда.



Адресация в C++

Программист может определить собственные переменные для хранения адресов областей памяти. Такие переменные называются *указателями*.

Указатель – именованный объект, предназначенный для хранения адреса области памяти (объекта, непоименованной области оперативной памяти либо точки входа в функцию).

Указатель не является самостоятельным типом, он всегда связан с каким-то другим типом. Указатели делятся на две категории:

- указатели на объекты;
- указатели на функции.

Эти категории указателей отличаются друг от друга свойствами и правилами манипулирования. Каждый *указатель* имеет соответствующий тип.

С указателями можно выполнять следующие *операции*:

- разыменование (*) – получение значения величины, адрес которой хранится в указателе;
- взятие адреса (&);
- присваивание;
- арифметические операции
  - сложение указателя только с константой,
  - вычитание: допускается разность указателей и разность указателя и константы,
  - инкремент (++) увеличивает значение указателя на величину sizeof(тип);

  - декремент (--) уменьшает значение указателя на величину sizeof(тип);
- сравнение;
- приведение типов.

Указатели одного и того же типа можно сравнивать с помощью стандартных операций сравнения. При этом сравниваются значения указателей, а не значения величин, на которые данные указатели ссылаются.

Вопросы для повторения темы 2

1. Укажите основные моменты представления данных на физическом уровне.
2. Как в памяти ЭВМ представляются целые числа без знака?
3. Как в памяти ЭВМ представляются целые числа со знаком?
4. Как в памяти ЭВМ представляются вещественные числа
5. Приведите классификацию базовых типов и структур данных. Кратко охарактеризуйте каждый класс.
6. Что такое встроенные типы данных?
7. Что такое уточняемые и перечисляемые типы данных? Для каких целей они применяются?
8. Перечислите конструируемые типы данных.
9. Что такое данные и структуры данных?
10. Что такое указательный тип данных?